

Competitive Programming

Section 58 Editorial: Full Virtual Contest Workflow

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
1.1	Read before code: problem dossiers	3
1.2	Problem-specific proof map	3
1.3	Detailed solution guide	3
2	A. Scoreboard Rank	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
3	B. Split Workload	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Quick MST Decision	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Read before code: problem dossiers

A. Scoreboard Rank. Reduce each row to $(-solved, penalty, name)$ and sort. That tuple is the stated tie rule; test exact ties and lexicographic names. Complexity $O(n \log n)$.

B. Split Workload. Every subset is one teammate's task set and its complement is the other's. Enumerating all masks covers every split; minimize the larger sum. Test one task and an exactly balanced split. Complexity $O(2^n)$ locally.

C. Quick MST Decision. Kruskal's accepted edges cross DSU components, so the cut property yields a minimum spanning tree. Test disconnected inputs and equal weights before interpreting savings.

1.2 Problem-specific proof map

A. Scoreboard Rank. The stored tuple $(-solved, penalty, name)$ is exactly the lexicographic contest ranking rule, so sorting those tuples gives the required order in $O(n \log n)$.

B. Split Workload. Every assignment of tasks to one teammate is one subset; enumerating all subsets therefore examines every two-way split and minimizes the larger load exactly.

C. Quick MST Decision. Kruskal's DSU rejects cycle edges and accepts only component-merging edges. Its cut-property proof gives the minimum network and therefore the requested savings decision.

1.3 Detailed solution guide

A. Scoreboard Sorting. Store each team by solved count, penalty, and name. The comparator sorts by more solved problems first, lower penalty second, and lexicographic name last. This mirrors the statement exactly, so once every team record is computed, sorting produces the required ranking.

B. Subset Partition Drill. The problem is a small knapsack/subset-sum decision. Maintain reachable sums after processing each item. Initially only sum 0 is reachable. For each value, update sums in descending order so the value is used at most once. The best partition is the reachable sum closest to half the total.

C. MST Decision Drill. Build the minimum spanning tree with Kruskal. If the MST weight is at most the budget, print feasible; otherwise not. The cut property ensures Kruskal gives the cheapest connected network, so no other spanning tree can fit the budget when the MST does not.

Common pitfalls. In contest workflow problems, the algorithms are often standard; the challenge is reading carefully, choosing the right template, and testing boundary cases before moving on.

These exercises are small contest-operation drills.

- A implements scoreboard ordering exactly: more solved first, lower penalty second, name as deterministic tiebreak.
- B is subset/partition search for balancing workload between two solvers.
- C recognizes MST savings quickly: total edge weight minus MST weight if the graph is connected.

The editorial emphasis is precision under time pressure: model the rule, test corner cases, then code the smallest reliable version.

2 A. Scoreboard Rank

2.1 Restatement

Sort teams by more solved, then lower penalty, then name.

2.2 Observations and algorithm

Store each team as `(-solved, penalty, name)` and sort tuples. This encodes the ranking rule directly; trace two teams tied on solves but not penalty, then a complete tie resolved by name.

2.3 Correctness proof

Tuple lexicographic order compares exactly the stated tie-break fields in their required priority, so sorted output is the requested scoreboard. $\square$

2.4 Complexity

Sorting is $O(n \log n)$ time and $O(n)$ memory. Avoid sorting names before negating solved count.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().splitlines()
    if not lines: return
    rows=[]
    for line in lines[1:]:
        name,sol,pen=line.split(); rows.append((-int(sol),int(pen),name))
    rows.sort(); print("\n".join(name for _,_,name in rows))
if __name__=="__main__": main()
```

3 B. Split Workload

3.1 Restatement

Split problem times between two solvers minimizing the maximum load.

3.2 Observations and algorithm

For each mask, one solver receives selected tasks and the other receives its complement. Enumerating every mask covers every split; update the answer with the smaller maximum load. Test one task and an exactly balanced subset.

3.3 Correctness proof

Every legal two-person allocation determines exactly one subset, and every mask determines one allocation, so taking the minimum examines the optimum. $\square$

3.4 Complexity

The local brute force is $O(2^n)$ time; it is intentionally limited to small n . Use a running subset sum rather than recomputing it in harder variants.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

3.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; a=data[1:]; best=10**9
    for mask in range(1<=n):
        s=sum(a[i] for i in range(n) if mask>>i&1); best=min(best,max(s,sum(a)-s))
```

```
    print(best)
if __name__=="__main__": main()
```

4 C. Quick MST Decision

4.1 Restatement

Use MST savings as a contest modeling drill.

4.2 Observations and algorithm

Sort edges and use DSU exactly as Kruskal: accepted edges merge components, rejected edges close cycles. Compare the resulting MST cost with the requested decision/budget only after checking connectivity.

4.3 Correctness proof

The cut property makes each component-merging minimum edge safe; therefore no connected subgraph can cost less than the constructed tree. $\square$

4.4 Complexity

Time $O(m \log m)$, memory $O(n)$. Test a disconnected graph and equal-weight edges; do not interpret a forest as a feasible network.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(ci
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; edges=[]; total=0; idx=2
    for _ in range(m):
        a,b,w=data[idx],data[idx+1],data[idx+2]; idx+=3; edges.append((w,a-1,b-1)); total+=w
```

```

p=list(range(n))
def f(x):
    while p[x]!=x: p[x]=p[p[x]]; x=p[x]
    return x
keep=0; cnt=0
for w,a,b in sorted(edges):
    ra,rb=f(a),f(b)
    if ra!=rb: p[ra]=rb; keep+=w; cnt+=1
print(total-keep if cnt==n-1 else -1)
if __name__=="__main__": main()

```

5 What to carry forward